

CHAPTER REVISION TOUR – 1

PYTHON

(Class 12)

INTRODUCTION TO PYTHON

Python is a high-level, interpreted, object-oriented programming language known for its simplicity and readability.

Features of Python:

- Easy to learn and use
- Interpreted language
- Platform independent
- Dynamically typed
- Supports Object Oriented Programming

```
1 print("Hello World")
```

Listing 1: Hello World Example

Exam Question:

Why is Python called an interpreted language?

Answer: Python executes code line by line without prior compilation.

VARIABLES

A variable is a named memory location used to store data.

Rules for naming variables:

- Must start with a letter or underscore (_)
- Cannot start with a digit
- No special characters except underscore
- Case sensitive

```
1 age = 18
2 name = "Anmol"
3 _marks = 95.5
```

Listing 2: Variable Examples

Dynamic Typing: Python allows changing data type at runtime.

```
1 x = 10
2 x = "Python" # Type changed from int to str
```

Listing 3: Dynamic Typing Example

DATA TYPES

Built-in Data Types in Python:

- int, float, complex
- str, bool
- list, tuple
- set, dict

Numeric Data Types

- `int` → Whole numbers

```
1 a = 10
2
```

- `float` → Decimal numbers

```
1 b = 3.14
2
```

- `complex` → Real + Imaginary

```
1 c = 2 + 3j
2
```

String Data Type

A string is a sequence of characters enclosed in quotes.

```
1 s = "Python"
2
3 # Indexing
4 s[0] # Returns 'P'
5
6 # Slicing
7 s[1:4] # Returns 'yth'
```

Listing 4: String Operations

MUTABLE & IMMUTABLE TYPES

Mutable Data Types: (Can be changed after creation)

- list
- set
- dictionary

Immutable Data Types: (Cannot be changed after creation)

- int, float
- string
- tuple

```
1 # Mutable Example
2 lst = [1, 2, 3]
3 lst[0] = 10      # Allowed - list is mutable
4
5 # Immutable Example
6 s = "Hello"
7 # s[0] = 'Y'      # Error - string is immutable
```

Listing 5: Mutability Examples

FLOW OF CONTROL

Flow of control determines the order in which statements are executed.

Types:

1. Sequential
2. Conditional
3. Looping

CONDITIONAL STATEMENTS

if statement

```
1 if x > 0:
2     print("Positive")
```

if-else statement

```
1 if x % 2 == 0:
2     print("Even")
3 else:
4     print("Odd")
```

if-elif-else

```
1 if marks >= 90:
2     grade = 'A'
3 elif marks >= 75:
4     grade = 'B'
5 else:
6     grade = 'C'
```

NESTED CONDITIONS

A nested if is an if statement inside another if statement.

```
1 if age >= 18:
2     if nationality == "Indian":
3         print("Eligible to vote")
```

Listing 6: Nested If Example

Important: Python uses indentation, not braces.

LOOPS

Loops are used to execute a block of code repeatedly.

Types of loops:

- while loop
- for loop

while loop

```
1 i = 1
2 while i <= 5:
3     print(i)
4     i = i + 1
```

Listing 7: While Loop Example

for loop

```
1 for i in range(1, 6):
2     print(i)
3
4 # range(start, stop, step)
5 for i in range(1, 10, 2):
6     print(i) # Prints 1, 3, 5, 7, 9
```

Listing 8: For Loop with range()

NESTED LOOPS

A loop inside another loop is called a nested loop.

```
1 for i in range(1, 4):
2     for j in range(1, 4):
3         print(i, j)
```

Listing 9: Nested Loop Example

Used in:

- Pattern printing
- Tables
- Matrices

JUMP STATEMENTS

break

Terminates the loop immediately.

```
1 for i in range(10):  
2     if i == 5:  
3         break  
4     print(i) # Stops when i reaches 5
```

continue

Skips the current iteration.

```
1 for i in range(5):  
2     if i == 2:  
3         continue  
4     print(i) # Skips printing 2
```

pass

Acts as a placeholder.

```
1 if x > 0:  
2     pass # Placeholder for future code
```

IMPORTANT PRACTICE QUESTIONS

1. Write a program to check whether a number is positive, negative or zero.
2. Print first 10 even numbers using a loop.
3. Find factorial of a number using while loop.
4. Display multiplication table of a given number.
5. Check whether a string is palindrome.

QUICK EXAM NOTES

- Python is indentation sensitive
- No semicolon required
- Variables are dynamically typed
- **break** exits loop completely
- **continue** skips current iteration
- Nested structures increase complexity

END OF TOUR – 1